INTERNAL TECHNICAL REFERENCE

Narration Studio Architecture and Internal Workings

End-to-end documentation of the Microsoft Agent Framework boilerplate that turns a voice transcript into a synced narration video. Written from the actual source in `E:\exercises\maf`.

Solution NarrationStudio.slnx

Project src/NarrationStudio/NarrationStudio.csproj · net10.0-windows

Audience Developers onboarding to the codebase

Classification Internal. Contains architecture and configuration details; API keys are not reproduced here.

Date 22 August 2026 · **Revision** 4

Status Confirmed against current source. Where behavior is implied by a library rather than this repo, that is labelled *inferred*.

Document control

Item	Value
Title	Narration Studio — Internal Technical Architecture and Workings
Scope	Entire application in this repository. One executable, two entry modes: CLI production and the HTTP job API for n8n.
Source of truth	C# source under `src/NarrationStudio`, `appsettings.json`, `n8n/`, `README.md`, and observed runtime behavior.
Out of scope	Microsoft Agent Framework library internals except as used here. ElevenLabs / Ollama / ffmpeg / n8n product internals except as called.
Secrets policy	This document names configuration keys but does not print secret values.
Companion notes	`docs/n8n-service.pdf`, `docs/memory-vs-mem0.pdf`, `docs/guardrails.pdf`, `docs/harness-engineering.pdf`, `docs/agentic-system.pdf`, `docs/orchestration-modes.pdf`

Revision 2 (21 August 2026) adds: Ollama one-sentence slide text; rephrase duration logs; timestamped agent-memory folders that are *not* wiped; snapshots in every mode; ElevenLabs key from `Studio:ElevenLabsApiKey`.

Revision 3 (22 August 2026) is the harness-centric MAF refactor: `ProductionHarness` owns the run; `DirectExecutor` replaces `DirectPipelineRunner`; `HarnessChecks` fails the process if artifacts are wrong; `events.jsonl`; `UseStubSlidePhrases`; option-driven prompts; cancellation through Ollama/TTS/ffmpeg; `AGENTS.md` and `tests/NarrationStudio.Tests`.

Revision 4 (22 August 2026) adds the n8n job service: empty args start Kestrel on `http://0.0.0.0:5088`; scoped pipeline DI; serial `JobWorker`; custom node + Form Trigger; Docker Compose mount into stack `n8n-api_icountera`. See §19 and `docs/n8n-service.pdf`.

> **HOW TO READ "CONFIRMED" VS "INFERRED"**
>
> **Confirmed** means the statement is directly visible in this repository's source or configuration. **Inferred** means the behavior comes from a NuGet package, OS API, or external service that this repo calls but does not implement. Those inferences are called out so a new developer does not treat library magic as local code.

Table of contents

1. **Application Overview**
2. **System Architecture**
3. **Detailed Component Breakdown**

4. Request/Response and Execution Flows

5. Database and Data Layer

6. API Documentation

7. Frontend / Internal UI Workings

8. Backend / Internal Service Workings

9. Security

10. Error Handling and Edge Cases

11. Configuration and Environment

12. Deployment and Infrastructure

13. Performance and Scalability

14. Testing

15. Developer Guide

16. Complete Dependency Map

17. Technical Diagrams

18. Appendix

19. n8n job service

1. Application Overview

1.1 Purpose and core functionality

Narration Studio is a **Windows console application** that produces a timestamped narration video from a source transcript. It is a Microsoft Agent Framework (MAF) boilerplate with a **harness-centric** loop: `ProductionHarness` owns run policy (stub copy, MAF or direct execution, failover, artifact checks). Specialist agents can drive the same production tools that `DirectExecutor` also calls. Agents do not own I/O; tools do.

The product contract, as implemented, is:

1. Read `voice_transcript.txt` (raw notes / transcript).
2. Rewrite it into a spoken ~7-minute talk, saved as `output/scripts/narration-rewritten.txt`. Today this rewrite is usually **stubbed** from `voice_transcript_ollama.txt` when `Studio:UseStubNarration` is true; otherwise it calls local Ollama.
3. Split that rewritten talk into `Studio:SegmentCount` narration scripts (default **8**).
4. Plan at most `Studio:MaxSlidesPerSegment` excerpts per script (default **5**). By default **Ollama rewrites each excerpt into one sentence** unless `Studio:UseStubSlidePhrases` is true. That sentence is drawn on the PNG. The original excerpt is kept as `SourcePhrase`.
5. Synthesize `audio-01.mp3` … `audio-NN.mp3` via ElevenLabs (if an API key is configured) or Windows SAPI + ffmpeg.
6. Sync each MP3 with that segment's slides into `video-01.mp4` … `video-NN.mp4`.
7. Merge into `video_yyyyMMdd_HHmmss.mp4` in the output folder.

The application is still a **Windows console factory**, not a multi-user SPA or SQL product. With **empty args** (or `--service`) the same EXE also hosts a small Kestrel job API on `http://0.0.0.0:5088` so Docker n8n can start productions. Passing `--orchestration` or a transcript path still runs the CLI and exits. There is no MVC site. The analogue of "controllers" is `JobEndpoints` (HTTP) plus the thin `ProductionApp` CLI; `ProductionHarness` still owns run policy; `ProductionTools` is still the only PNG/MP3/MP4 surface. See §19.

1.2 Main use cases

Use case	How it is invoked	What happens
Produce a video from the repo transcript (typical)	`dotnet run --project src\NarrationStudio -- --orchestration direct` from the repo root	Clears previous media, copies stub narration, segments, Ollama slide sentences (unless stubbed), PNGs, TTS, compose, merge. Writes timestamped memory snapshots including `events.jsonl`. Then `HarnessChecks` verifies artifacts.
Agent-orchestrated production	Omit `--orchestration` (defaults to `handoff`) or pass `sequential`	`ProductionHarness` builds MAF agents; specialists call the same tools. If TryCreate fails or the merged file is missing afterward, `DirectExecutor` finishes the job.

		Unknown `--orchestration` values are treated as handoff.
Use a different transcript file	Pass a path as the first non-flag argument, or set `Studio:TranscriptPath`	`ProductionApp.ResolveTranscriptPath` walks a candidate list until a file exists.
Call Ollama for the rewrite instead of the stub	Set `Studio:UseStubNarration` to `false`	`NarrationRewriteService.RephraseAsync` posts the raw transcript to Ollama.
Skip Ollama for on-screen slide sentences	Set `Studio:UseStubSlidePhrases` to `true`	`SlidePhraseRewriteService` copies each excerpt into `Phrase` (no HTTP). Independent of narration stubbing.
Prevent overlapping runs	Automatic	Named mutex `NarrationStudio.Production`. CLI second instance throws. HTTP jobs queue one at a time and take the same mutex.
Start a production from n8n	Empty-args service + Form Trigger workflow	n8n uploads files, POST `/v1/jobs` returns 202, polls until complete, downloads the merged MP4. Artifacts under `output/jobs/{jobId}/`.

1.3 High-level architecture

Composition starts in `Program.cs`. **CLI** uses Generic Host (`Host.CreateApplicationBuilder`) and calls `ProductionApp.RunAsync`. **Service** uses `WebApplication`, maps `JobEndpoints`, and runs `JobWorker`. Pipeline types are registered **scoped** (`AddNarrationStudioPipeline`) so each HTTP job can overlay `Studio:` / `Model:` without mutating another run. Control flows: host → CLI `ProductionApp` or HTTP `JobWorker` → `ProductionHarness` → MAF `WorkflowHost` or `DirectExecutor` → `ProductionTools` → domain services → disk. After tools finish, `HarnessChecks` verifies artifacts or throws.



Figure 1. Logical layers. Confirmed: all boxes correspond to types or processes named in this repo.

1.4 Overall data and control flow

Data is a single in-memory `PipelineState` cloned on read via JSON round-trip, persisted after every mutation to `output/pipeline-state.json`. Artifacts are files. There is no message bus.

Control flow for a typical current run (`UseStubNarration=true` , `--orchestration direct`):

1. `Program.cs` treats `--orchestration` / a transcript path as CLI and calls `ProductionApp.RunAsync(args)` . Empty args start the job API instead (§19).
2. Mutex acquired. Transcript and output directory resolved. `PipelineStore.Initialize` deletes previous **media** (scripts/slides/audio/video and `video_*.mp4`) but **keeps** prior `output/agent-file-memory/production-notes-*` folders. It creates a new timestamped memory folder and writes `run.txt` .
3. Banner includes `Harness : MAF+tools` , TTS provider, optional stub lines, and `Memory : {full path}` .

4. `ProductionHarness.RunAsync` copies stub narration if configured, then runs `DirectExecutor` (or MAF, with failover to direct if TryCreate fails or merge is missing). Host `ApplicationStopping` is passed through.
5. Direct mode always re-segments, plans slides (Ollama sentences unless `UseStubSlidePhrases`), re-renders PNGs, re-synthesizes, re-composes, and merges. Each tool call appends `events.jsonl`. Each `Mutate` refreshes `pipeline-status.txt` and `notes.txt`.
6. `HarnessChecks.Verify` requires `status == complete`, expected script count, slides ≤ max, audio/video files, and a merged path on disk. Failure throws `HarnessCheckException` (non-zero exit) even if an agent said Done.
7. Console prints `Done: {merged path}`.

Status strings written onto state: `created` → `narration-ready` → `scripts-ready` → `slides-planned` → `slides-rendered` → `audio-ready` → `videos-ready` → `complete`.

2. System Architecture

2.1 Frontend, backend, database, APIs, services, external dependencies

Map the requested web-app vocabulary onto what actually exists:

Typical web layer	Narration Studio equivalent	Confirmed location
Frontend	Console stdout/stderr. No UI framework, routing, or components.	`ProductionApp`, `WorkflowHost.RunAsync`, `DirectExecutor`
Backend	The same process. Business logic is C# services invoked in-process.	`Harness`, `Services`, `Tools`, `Orchestration`
Database	None. Persistence is JSON + files under `output/`.	`PipelineStore`, `Models/PipelineModels.cs`
HTTP API (this app)	Job API when started with empty args: `/health`, `/v1/jobs`, status, events, merged MP4. The process is also a client of Ollama (unless both stub flags are on) and ElevenLabs.	`Service/JobEndpoints.cs`, rewrite/TTS services
Internal API	`ProductionTools` methods exposed to agents via `AIFunctionFactory.Create`	`Tools/ProductionTools.cs`
External processes	`ffmpeg`, `ffprobe` on PATH	`FfmpegProcess` in `TtsService.cs`

2.2 Responsibilities of each layer

Layer	Responsibility	Must not
Program.cs	Configuration providers, DI, CLI vs Kestrel branch	Contain pipeline logic
ProductionApp	Mutex, path/mode resolution, banner, catch harness/pipeline failures	Call tools, choose MAF vs direct, verify artifacts
ProductionHarness	Stub copy, MAF vs DirectExecutor, merge failover, HarnessChecks	Draw PNGs or run ffmpeg
WorkflowHost / AgentFactory	Build MAF agents and workflows; stream agent text to the console	Write media files themselves
DirectExecutor	Deterministic ordered tool calls; overwrite-oriented regeneration	Choose agent graph
HarnessChecks / HarnessEvents	Fail the process if artifacts are wrong; append `events.jsonl`	Produce media
ProductionTools	Validate preconditions, call services, mutate PipelineStore, return short status strings; only PNG/MP3/MP4 I/O	Know ffmpeg flags
Domain services	Pure-ish transforms and I/O (rewrite, split, PNG, TTS, video)	Choose orchestration mode
PipelineStore	Thread-safe state, filenames, wipe-on-start, persist JSON, agent status summary	Call external APIs
Guardrails / context / file memory	Filter agent text, inject status into prompts, optional on-disk agent notes	Drive the media pipeline

2.3 How the layers communicate

All communication is **in-process method calls** through constructor-injected singletons. Agents do not get a private copy of services; they receive `AIFunction` wrappers around the same `ProductionTools` instance. That is why a tool called by an agent updates the same `PipelineStore` that DirectExecutor later reads.

External communication:

- **Ollama** — HTTP via `OllamaApiClient` (OllamaSharp) at `Model:Endpoint`, default `http://localhost:11434`.
- **ElevenLabs** — HTTP POST `https://api.elevenlabs.io/v1/text-to-speech/{voiceId}` with header `xi-api-key`.
- **ffmpeg / ffprobe** — child processes, stdout/stderr captured asynchronously to avoid deadlock, 8-minute timeout.
- **Windows SAPI** — `System.Speech.Synthesis.SpeechSynthesizer` writing WAV, then ffmpeg to MP3.

2.4 Why this shape

- **Harness owns the loop.** Agents are optional drivers. Direct mode exists so production still works when the LLM is slow, missing, or poorly follows handoff instructions. Confirmed in `ProductionHarness`: `direct` mode, failed `TryCreate`, or a missing merged file always falls through to `DirectExecutor`. After tools finish, `HarnessChecks` can still fail the process.
- **Wipe media on start; keep memory folders.** Scripts, slides, audio, video, and leftover `video_*.mp4` are deleted so a rerun cannot stitch old media. Agent memory is **not** deleted: each run uses `output/agent-file-memory/production-notes-{RunId}/`.
- **Stub rewrite.** Local Ollama rewrite is long-running (timeout 30 minutes). Stubbing lets the rest of the factory be developed against a known talk.
- **Windows TFM.** `net10.0-windows` is required for `System.Speech`.

3. Detailed Component Breakdown

This section covers every significant type in the project. Small helpers are included because they affect production reliability.

3.1 Program.cs — composition root

Purpose. Choose CLI vs job service, bind options, register the pipeline, run. There is no `Main` class; top-level statements are used.

CLI vs service (confirmed). `IsProductionCli` is true if args contain `--cli`, `--orchestration`, or a non-flag token (transcript path). Then Generic Host runs `ProductionApp` and exits. `--service` forces the web host. Empty args start Kestrel on `Service:ListenUrl` (default `http://0.0.0.0:5088`), register `JobStore` / `JobQueue` / `JobWorker`, and map `JobEndpoints`.

Config loading (confirmed, and easy to get wrong). Launch profile `workingDirectory` is the repo root (`$(ProjectDir)..\..`), so the first `appsettings.json` lookup is `{cwd}/appsettings.json`, which does **not** exist. The second explicit call loads `AppContext.BaseDirectory/appsettings.json` — the copy MSBuild places next to the DLL. That is the file that actually configures a `dotnet run` from the repo root.

Inputs / outputs. Input: `args`, JSON files, environment variables. CLI output: `ProductionApp.RunAsync`. Service output: HTTP on 5088 until Ctrl+C.

Pipeline registrations live in `Hosting/StudioServiceCollection.AddNarrationStudioPipeline`. Runtime options, tools, harness, agents, and `ProductionApp` are **scoped** so each job can overlay `Studio:` / `Model:`. A few pure helpers stay singleton (segmenter, slide planner, PNG renderer, video composer).

Error handling. CLI failures bubble from `RunAsync` and crash the process. Service JSON parse errors return 400; job failures are stored on the job record.

3.2 ProductionApp

File: `src/NarrationStudio/ProductionApp.cs`

Responsibilities. Thin CLI: exclusive run (mutex), resolve transcript/output/orchestration, initialize store (wipes media, creates timestamped memory folder), print a banner (including `Harness : MAF+tools`, TTS provider, stub flags, `Memory` path), write `run.txt`, call `ProductionHarness.RunAsync`, print Done. It does **not** choose MAF vs direct, copy the stub, or verify artifacts — those live on the harness.

Important methods.

Method	Behavior
`RunAsync(string[] args)`	CLI lifecycle in §4. Catches `HarnessCheckException` separately from other pipeline failures.
`ResolveTranscriptPath`	Candidates: first non-`--` arg, then `Studio:TranscriptPath`, then several relative `voice_transcript.txt` locations. First existing file wins. Else `FileNotFoundException`.
`ResolveOutputDirectory`	`Studio:OutputDirectory` or `output`, rooted against current directory.
`ResolveOrchestration`	`--orchestration=x` or `--orchestration x`. Default `handoff`. Values other than `direct` and `sequential` (including typos) take the handoff branch in the harness.

Auth. None. Process identity is the logged-in Windows user.

3.2b ProductionHarness

File: `src/NarrationStudio/Harness/ProductionHarness.cs`

Purpose. The run loop. Confirmed sequence:

1. If `UseStubNarration`, call `RephraseTranscriptIntoNarration` **before** `AgentFactory.TryCreate`, so stubbing does not look like an Ollama rewrite.
2. If orchestration is `direct`, or `TryCreate` fails, run `DirectExecutor.RunAsync(ApplicationStopping)`.
3. Else build sequential or handoff workflow, run it with the same cancellation token, then if `MergedVideoPath` is empty or the file is missing, log a warning and run DirectExecutor.
4. Always call `HarnessChecks.Verify()`. A successful-looking agent transcript still fails the process if artifacts are wrong.

Handoff and sequential are **MAF**. Direct, CLI, failover, and this class are **this app**.

3.2c HarnessEvents and HarnessChecks

HarnessEvents (`Harness/HarnessEvents.cs`) appends one JSON line per tool to `output/agent-file-memory/production-notes-{RunId}/events.jsonl`: timestamp, name, elapsedMs, ok, error. Tools wrap work in `Trace` / `TraceAsync`. If output is not initialized yet, writes are skipped.

HarnessChecks (`Harness/HarnessChecks.cs`) is a static-capable verifier used after the run and in unit tests. Failures throw `HarnessCheckException` listing:

- status is not `complete`

- segment count ≠ `Studio:SegmentCount`
- any segment has more slides than `MaxSlidesPerSegment`
- audio or video path missing or file absent
- merged video path missing or file absent

This is the process-level Done gate. Agents cannot skip it.

3.3 PipelineStore and PipelineState

Files: `Services/PipelineStore.cs` , `Models/PipelineModels.cs`

Purpose. Single source of pipeline truth for one process run. Analogous to a unit-of-work + repository, but for files.

Internal structure. Private `PipelineState _state` plus `object _gate` . `Snapshot()` serializes then deserializes with source-generated `PipelineJsonContext` so callers cannot mutate live state without `Mutate` . Every `Mutate` writes `output/pipeline-state.json` .

Initialize. Sets transcript and output paths, creates directories, **deletes** previous scripts/slides/audio/video, leftover `video_*.mp4` , and old `pipeline-state.json` . It does **not** delete `agent-file-memory` . It creates `AgentMemoryDirectory` = `output/agent-file-memory/production-notes-{RunId}` and writes snapshots there on every persist. Returns the count of deleted media files for the banner.

Naming helpers (confirmed).

```
audio-{nn}.mp3
video-{nn}.mp4
segment-{nn}-slide-{nn}.png
narration-rewritten.txt
video_{RunId}.mp4   where RunId = yyyyMMdd_HHmmss at store construction
production-notes-{RunId}\   under output\agent-file-memory\
```

State fields. See §5. `RunId` is assigned in the constructor **before** Initialize; wiping old merged videos uses glob `video_*.mp4` , not the new RunId.

SummarizeForAgent. Counts scripts/slides/images/audio/videos from in-memory segments plus File.Exists. “Next required step” is a priority chain. After wipe, in-memory segments are empty even if a human dropped files on disk mid-run — agents are steered by this summary, not by scanning leftover files (except existence checks on paths already recorded in state).

Concurrency. Lock around snapshot/mutate/initialize. Tool methods sometimes snapshot, work, then mutate — a second thread could interleave, but the app is effectively single-threaded aside from ffmpeg async reads.

Why JSON clone for Snapshot. Cheap isolation without implementing `ICloneable` on a graph of lists. Cost is negligible at this scale.

3.4 ProductionTools

File: `Tools/ProductionTools.cs`

Purpose. The internal production API — the only place that writes PNG/MP3/MP4 (plus script text). Each public method has a `[Description]` consumed by MAF when wrapping as an `AIFunction`. Direct mode calls the same methods. Descriptions interpolate option names (`Studio:SegmentCount`, `Studio:MaxSlidesPerSegment`, `Studio:TargetTalkMinutes`) rather than hardcoding 8 / 5 / 7.

Methods, inputs, outputs.

Method	Input	Writes	State status	Throws if
`GetPipelineStatus`	—	—	—	—
`LoadTranscript`	—	—	—	file missing
`LoadNarrationSource`	—	—	—	no NarrationPath file
`RephraseTranscriptIntoNarration`	raw transcript via LoadTranscript; rewrite service may ignore it when stubbing	`scripts/narration-rewritten.txt`	narration-ready	stub missing/short; Ollama empty
`SegmentTranscriptIntoEightScripts`	rewritten text	`narration-01..08.txt`, `narration-all.txt`	scripts-ready	count ≠ SegmentCount
`SaveNarrationScripts`	JSON array of segments	same script files	scripts-ready	parse fail; wrong count
`PlanSlidesForAllSegments`	each segment.Script	SlideSpec lists; Ollama one-sentence `Phrase`	slides-planned	scripts missing; Ollama fail
`RenderAllSlideImages`	slide `Phrase` (Ollama sentence)	PNGs; always overwrite	slides-rendered	scripts missing
`SynthesizeAllAudioAsync`	each script	MP3s; duration probed	audio-ready	scripts missing; TTS/ffmpeg fail
`ComposeAllSegmentVideos`	audio + images	segment MP4s	videos-ready	audio/slides missing
`MergeAllVideos`	8 video paths	merged MP4	complete	count ≠ SegmentCount

EnsureScripts. Requires `Segments.Count == SegmentCount` in memory — not a directory listing. After a wipe, agents that skip segmentation cannot render.

Cancellation. Tools capture `IHostApplicationLifetime.ApplicationStopping` and pass it into Ollama rewrite, slide-phrase rewrite, TTS/ElevenLabs, ffprobe, and ffmpeg. DirectExecutor and WorkflowHost receive the same token from the harness. Ctrl+C can abort in-flight HTTP and child processes. Confirmed: tools call `ThrowIfCancellationRequested` around PNG render as well.

Events. Every public tool method is wrapped in `HarnessEvents.Trace` / `TraceAsync`.

3.5 NarrationRewriteService and OllamaChatClientFactory

File: `Services/NarrationRewriteService.cs` (both types)

OllamaChatClientFactory. Lazy singleton `IChatClient` (`OllamaApiClient`) with a .NET `Lock`. HttpClient BaseAddress = endpoint, Timeout = max(1, TimeoutMinutes) minutes. Used both for rewrite and, when no Azure/OpenAI key, for agent orchestration.

RephraseAsync.

- If `UseStubNarration`: resolve stub path (transcript dir, cwd, BaseDirectory, and a walk-up from bin). Strip `<think>` and fences. Reject if length < 400 characters. Does not send the raw transcript to a model.
- Else: system prompt asks for ~ `TargetTalkMinutes * 140` words, spoken voice, no invented facts. User message is the raw transcript. Temperature 0.35. Same length check on the result. Logs elapsed seconds: `Received rewritten narration (N words) in Xs.`

Why 400 characters. Heuristic to catch empty or “thinking only” model output. Not a linguistic quality gate.

3.6 TranscriptSegmenter

Splits on `(?<=[\.!\?])\s+`, then packs sentences into `SegmentCount` buckets by character budget. If too few groups, it peels sentences from the richest bucket. Titles are “Segment NN — {first sentence truncated to 72 chars}”.

WriteScriptFiles overwrites `narration-{nn}.txt` and concatenates `narration-all.txt`.

Edge case (confirmed). If every remaining group has a single sentence and count is still below 8, the loop can `break` after duplicating one sentence into a new group, then `Take(buckets)` may still return fewer than 8. `SegmentTranscriptIntoEightScripts` would then throw. Unlikely for a 7-minute talk; possible for a tiny stub.

3.7 SlidePlannerService and SlidePhraseRewriteService

SlidePlannerService clamps max slides to 1..5. Sentence-splits the script; if more sentences than max, concatenates chunks. Each item gets `SourcePhrase` and an initial `Phrase` (trimmed to 220 characters). `ImagePath` is filled later by render. This split is deterministic regex, not an LLM.

SlidePhraseRewriteService (`Services/SlidePhraseRewriteService.cs`) then, for every slide that is not yet `PhraseRewritten`:

- If `UseStubSlidePhrases` : copy `SourcePhrase` into `Phrase` (no HTTP). Independent of `UseStubNarration` .
- Else: call local Ollama. The model must return exactly one short sentence (about 8–22 words). Thinking tags and fences are stripped via `NarrationRewriteService.StripModelExtras` . Temperature 0.2. Empty Ollama output falls back to the source excerpt with a warning.

The sentence is stored in `Phrase` ; the PNG renderer draws `Phrase` only. Live Ollama rewrites log `Rephrased slide NN-MM in Xs: …` and a batch total. Stub slides do not need Ollama.

`PlanSlidesForAllSegments` and `RenderAllSlideImages` are both async and both call `RewriteAllAsync` . Render skips slides already marked rewritten so the same excerpt is not billed twice in one run.

3.8 SlideImageRenderer

ImageSharp PNG, 1920×1080 by default. Dark navy canvas, gold rule, header “SEGMENT nn · SLIDE x/y”, wrapped body text, footer = segment title. Fonts tried: Segoe UI, Arial, Calibri, else first system family.

WriteAllBytesWithRetry. FileMode.Create with share ReadWrite|Delete, 6 attempts, then write a GUID sidecar and copy over. Exists because Windows file locks from ImageSharp, explorers, or leftover ffmpeg were observed during development.

3.9 TtsService and FfmpegProcess

Key resolution. `Studio:ElevenLabsApiKey` first, else env `ELEVENLABS_API_KEY` . If present: POST MPEG to ElevenLabs; on non-success throw with status and body (no fallback to SAPI). If absent: SAPI WAV, or NAudio sine “tone bed” if SAPI throws, then ffmpeg libmp3lame.

DescribeProvider is used in the startup banner so operators can see ElevenLabs vs SAPI without reading logs.

FfmpegProcess. Starts ffmpeg/ffprobe with redirected stdout and stderr **both read asynchronously**, WaitForExit loop with 8-minute deadline, kill process tree on timeout or `CancellationToken` . This avoids the classic deadlock of reading one stream to end before the other. Probe returns stdout; run returns stderr (unused except on failure).

Duration. ffprobe format duration; parse fail defaults to 5 seconds (would distort slide timing if probe fails).

3.10 VideoComposerService

ComposeSegment. Writes ffmpeg concat demuxer list: each slide `file` + `duration` = max(0.4, audioDuration / slideCount), last file repeated (ffmpeg concat still-image requirement). Mixes concat video with the MP3, scale/pad to configured size, fps, libx264 stillimage, aac, `-shortest` , faststart. `-y` overwrites.

Merge. Concat list of the eight MP4s. Tries stream copy first; on any exception re-encodes libx264+aac. That fallback is why mismatched timebases from vfr concat can still produce a playable master.

Path escaping for concat files: backslash → slash, single quote → ffmpeg concat escape.

3.11 DirectExecutor

File: `Orchestration/DirectExecutor.cs` (renamed from `DirectPipelineRunner`). Reference executor and failover path. Logs, prints status, rephrases only if `NarrationPath` is empty/missing (so the harness stub copy is not duplicated unless something deleted the file), then **always** segments, plans, renders, synthesizes, composes, merges. No skip-if-exists on media. Honors `CancellationToken` at the start and before segmentation. Return value is the merge tool's string; ProductionApp reads `MergedVideoPath` from state after the harness returns.

3.12 AgentFactory and ProductionCrew

ProductionCrew is a record of seven `AIAgent` instances.

TryCreate. Builds one `IChatClient`: Azure OpenAI if endpoint set (key or DefaultAzureCredential), else OpenAI if `OPENAI_API_KEY`, else Ollama. Failure returns false → ProductionHarness uses DirectExecutor.

Each agent: `chatClient.AsAIAgent(ChatClientAgentOptions)` with id/name/description, instructions, tools, and two context providers. Then `.AsBuilder().Use(guardrails.RunAsync).Use(guardrails.FunctionAsync).Build()`.

Instructions and handoff text interpolate `SegmentCount`, `MaxSlidesPerSegment`, and `TargetTalkMinutes` (plus `StudioNaming` audio/video ranges). Confirmed: prompts are no longer hardcoded to "8 scripts".

File memory: MAF `FileMemoryProvider` + `FileSystemAgentFileStore` under `output/agent-file-memory`, working folder `production-notes-{RunId}`. That provider is constructed only inside `TryCreate`, so **direct mode never starts it**. **Inferred:** MAF may write extra files if agents use the file store during handoff/sequential.

Independently, `PipelineStore` writes `run.txt`, `pipeline-status.txt`, and `notes.txt` into the same timestamped folder on every persist, including direct mode. Older `production-notes-*` folders are kept. See also `docs/memory-vs-mem0.pdf`.

Chat client priority is Azure → OpenAI → Ollama. A developer with Azure env vars set will **not** use local Ollama for specialists even if Ollama is running. Confirmed in `CreateChatClient`.

3.13 WorkflowHost

Handoff. `AgentWorkflowBuilder.CreateHandoffBuilderWith(coordinator)`. If stubbing, rewriter is omitted from the graph. Specialists hand back to coordinator. Coordinator has autonomous mode, turnLimit 8, continuation prompt to GetPipelineStatus. **Inferred:** MAF runtime interprets handoff conditions as natural-language routing hints to the model, not compiled predicates.

Sequential. `BuildSequential` with rewriter omitted when stubbing — because the harness already wrote the talk, not ProductionApp.

RunAsync. `InProcessExecution.RunStreamingAsync`, sends `TurnToken(emitEvents: true)`, prints `AgentResponseUpdateEvent` text grouped by executor id, returns on `WorkflowOutputEvent`.

BuildUserPrompt. Injects stub vs Ollama rewrite step plus live `SummarizeForAgent`.

3.14 ProductionGuardrails

Inbound user messages containing blocked terms (`harmful` , `illegal` , `violence` , `exploit` , `malware` , `bypass paywall`) are replaced with a blocked marker; the run short-circuits with a refusal. All messages get email and `###-###-####` phone redaction.

FunctionAsync logs tool start/finish only. The previous "count the word slide and warn" path is **gone**. Script and slide caps are enforced in tools (throw) and again in `HarnessChecks` (fail the process). Guardrails apply only to agent runs, not DirectExecutor. Fuller write-up: `docs/guardrails.pdf` .

3.15 ProductionContextProvider

On invoke, injects instructions: current summary, exact file naming, segment/slide caps, stay on task. After invoke, appends truncated assistant text to `PipelineState.Notes` (max 24, drop oldest). Notes are persisted in `pipeline-state.json` and copied to the timestamped memory `notes.txt` . They are **not** injected back into `ProvideAIContextAsync` (audit log, not retrieval).

3.16 Configuration types

`StudioOptions` section `Studio` ; `OllamaOptions` section `Model` . Defaults in the class match appsettings for segment counts, stubbing, voice id, video size. Options are captured at construction (`IOptions<T>.Value`), so `reloadOnChange` on JSON files does **not** update live option objects already injected. Confirmed: services store `_options = options.Value` .

3.17 Launch profile

`Properties/launchSettings.json` profile `NarrationStudio` : empty args, working directory repo root. That is why transcripts at repo root are found and output is `E:\exercises\maf\output` rather than under `bin` .

4. Request/Response and Execution Flows

There are no inbound HTTP requests. "Request" here means a developer invoking the executable.

4.1 Direct orchestration (confirmed current happy path)

```
User: dotnet run --project src\NarrationStudio -- --orchestration direct
  → Host.CreateApplicationBuilder
  → Bind Studio + Model options
  → ProductionApp.RunAsync
      → Mutex NarrationStudio.Production
      → Resolve voice_transcript.txt (repo root)
      → Resolve output/ (cwd-relative)
      → orchestration = "direct"
      → PipelineStore.Initialize  (delete previous media; keep old memory folders)
      → create output\agent-file-memory\production-notes-{RunId}\
      → Banner: Transcript, Output, Mode, Harness : MAF+tools, Previous, Rewrite stub, Slides stub (if
on), TTS, Memory path
      → WriteRunHeader → run.txt
      → ProductionHarness.RunAsync
            → UseStubNarration → ProductionTools.RephraseTranscriptIntoNarration
                  → LoadTranscript (raw file, unused by stub rewrite)
                  → NarrationRewriteService.RephraseAsync → read stub, strip think tags
                  → write narration-rewritten.txt, status=narration-ready
            → DirectExecutor.RunAsync(ApplicationStopping)
                  → skip rephrase (NarrationPath exists)
                  → SegmentTranscriptIntoEightScripts
                  → PlanSlidesForAllSegments
                        → regex split ≤ MaxSlidesPerSegment
                        → Ollama one sentence per excerpt unless UseStubSlidePhrases
                  → RenderAllSlideImages  (PNG uses Phrase, not SourcePhrase)
                  → SynthesizeAllAudioAsync  (N sequential ElevenLabs or SAPI calls)
                  → ComposeAllSegmentVideos
                  → MergeAllVideos
            → HarnessChecks.Verify  (throws if artifacts wrong)
      → Console: Done: output\video_{RunId}.mp4
```

4.2 Handoff orchestration

1. Same init. Stub rewrite happens inside `ProductionHarness` before TryCreate.
2. `AgentFactory.TryCreate` — if this returns false, jump to DirectExecutor.
3. `CreateHandoffWorkflow` then `RunAsync` with user prompt and `ApplicationStopping`.
4. Coordinator should call `GetPipelineStatus` and hand off. Specialists call tools that mutate store and disk (and append events.jsonl).
5. Autonomous continuation up to 8 coordinator turns *(inferred MAF semantics)*.
6. If `MergedVideoPath` missing or file absent → warning log → DirectExecutor. Direct will not skip media generation, so a partial agent run is overwritten into a complete video (scripts already in memory if agents segmented; if not, direct segments from rewritten file).
7. `HarnessChecks.Verify` always runs.

Subtlety. Direct after a partial agent run still requires in-memory `Segments` or it will call `SegmentTranscriptIntoEightScripts` from the rewritten file. Agent-authored scripts in memory are kept for that process; they are not reloaded from disk on a new process because Initialize wipes and does not hydrate from leftover JSON (JSON was also deleted).

4.3 Sequential orchestration

Same as handoff but `BuildSequential` runs specialists in a fixed list. Stubbing removes the rewriter from that list because the harness already wrote the talk.

4.4 ElevenLabs request/response (external)

Item	Value (confirmed in TtsService)
Method / URL	POST https://api.elevenlabs.io/v1/text-to-speech/{ElevenLabsVoiceId}
Headers	`xi-api-key` , `Accept: audio/mpeg`
JSON body	`{ "text": "<segment script>", "model_id": "<ElevenLabsModelId>" }`
Success	Body copied to audio-NN.mp3
Failure	Exception including HTTP status and response body; pipeline stops; no SAPI fallback

4.5 Ollama rewrite request (when stubbing off)

Via Microsoft.Extensions.AI `GetResponseAsync` on `OllamaApiClient` . **Inferred wire format:** OllamaSharp talks to Ollama's HTTP API (typically `/api/chat`). This repo does not construct that JSON manually. Timeout is the HttpClient timeout (default 30 minutes).

4.6 Frontend-to-backend-to-database

Does not apply. Console is the UI; services are in-process; "database" is `pipeline-state.json` written on Mutate.

5. Database and Data Layer

There is **no database server, ORM, or migration framework**. Documented here is the file data layer that plays the same role.

5.1 Architecture

One JSON document plus a directory tree. Schema is the C# types in `PipelineModels.cs` , serialized with `System.Text.Json` source generation, camelCase, indented.

5.2 “Tables” (files and collections)

Store	Purpose	Lifetime
`output/pipeline-state.json`	Run metadata, segments, notes, paths	Rewritten every Mutate; deleted at next Initialize
`output/scripts/*`	Rewritten talk and 8 scripts	Deleted at next run start
`output/slides/*.png`	Slide images	Deleted at next run start
`output/audio/*.mp3`	TTS	Deleted at next run start
`output/video/*.mp4` and concat txt	Segment videos + ffmpeg lists	Deleted at next run start
`output/video_*.mp4`	Merged masters	All matching files deleted at next run start
`output/agent-file-memory/production-notes-{RunId}/`	run.txt, pipeline-status.txt, notes.txt, events.jsonl; optional MAF files in handoff	New folder each run; prior folders kept

5.3 Relationships

PipelineState 1—* NarrationSegment 1—* SlideSpec. Paths on segments/slides are denormalized strings, not foreign keys. Integrity is “file exists at path” checks in tools and SummarizeForAgent.

5.4 Schema (important fields)

Type	Fields
PipelineState	runId, transcriptPath, narrationPath, outputDirectory, startedAt, status, segments[], mergedVideoPath, notes[]
NarrationSegment	number, title, script, slides[], audioPath, videoPath, audioDurationSeconds
SlideSpec	number, sourcePhrase, phrase (Ollama one-sentence), phraseRewritten, imagePath

5.5 Queries and access

No query language. Access is LINQ over in-memory lists inside locked Mutate/Snapshot. There is no index. Hydration from a previous JSON file is **not implemented**; Initialize always starts empty segments after wipe.

5.6 Validation and integrity

- Segment count must equal `Studio:SegmentCount` (default 8) at segment and merge boundaries.

- Slide lists trimmed to MaxSlidesPerSegment at render time.
- Stub/Ollama narration must be ≥ 400 characters.
- JSON save of agent scripts uses `PipelineJsonContext.ListNarrationSegment`.
- No checksums, no transactional disk writes (JSON write is not atomic replace; a crash mid-write can corrupt pipeline-state.json). Next run deletes it anyway.

5.7 Caching

Ollama client instance is cached for process lifetime. No artifact cache: reruns recompute everything. ImageSharp fonts are loaded per render call (minor cost).

6. API Documentation

This application **does not expose HTTP endpoints**. The internal API is `ProductionTools`. Agents see the `[Description]` strings. Below is the contract a developer or agent relies on.

6.1 Authentication

None on tools. ElevenLabs and optional OpenAI/Azure keys authenticate **outbound** calls only.

6.2 Tool catalog

Tool	Sync	Arguments	Return (string)	Errors
GetPipelineStatus	sync	none	multi-line status	none
LoadTranscript	sync	none	raw file text	FileNotFoundException
LoadNarrationSource	sync	none	rewritten talk	InvalidOperationException
RephraseTranscriptIntoNarration	async	none	"Saved N-word narration to {path}."	FileNotFound / InvalidOperation / HTTP timeout
SegmentTranscriptIntoEightScripts	sync	none	"Saved N narration scripts to {dir}."	count mismatch; empty transcript
SaveNarrationScripts	sync	segmentsJson	"Saved N agent-authored…"	JSON / count
PlanSlidesForAllSegments	async	none	planned count (Ollama or stub sentences)	EnsureScripts; Ollama unless stub slides

RenderAllSlideImages	async	none	slides directory	EnsureScripts; IOException on PNG lock; Ollama if not yet rewritten
SynthesizeAllAudioAsync	async	none	audio directory	EnsureScripts; ElevenLabs HTTP; ffmpeg
ComposeAllSegmentVideos	sync	none	video directory	missing audio/slides; ffmpeg
MergeAllVideos	sync	none	"Merged video: {path}"	need 8 videos; ffmpeg

`SaveNarrationScripts` payload: JSON array of objects with `number`, `title`, `script` (camelCase via source-gen). Extra slide/audio fields may deserialize but are unused until later tools fill them.

6.3 CLI "API"

```
NarrationStudio.exe                          # job API on :5088 (also --service)
NarrationStudio.exe [transcriptPath] [--orchestration handoff|sequential|direct]
NarrationStudio.exe --orchestration=direct
```

Unknown flags are ignored except they are not treated as transcript paths if they start with `--`. Confirmed: no `--help`. Empty args are the n8n service, not a default handoff CLI run.

6.4 Error responses

Tools throw .NET exceptions. MAF *inferred* surfaces those as tool errors to the model. Direct/harness CLI: `ProductionApp` logs `Production pipeline failed` or `Harness checks failed` and rethrows → process crash. Job API: create-parse failures are 400 JSON `{ error, detail }`; a failed production sets job `status=failed` rather than crashing Kestrel.

6.5 Job HTTP API (n8n)

Served only in service mode. Full operator steps and Docker Compose YAML: `docs/n8n-service.pdf` and §19.

Method	Path	Notes
GET	`/health`	No API key
POST	`/v1/jobs`	JSON or multipart. **202** `{ jobId, status: queued }`
GET	`/v1/jobs/{id}`	`queued` / `running` / `complete` / `failed`
GET	`/v1/jobs/{id}/events`	`events.jsonl` lines
GET	`/v1/jobs/{id}/artifacts/merged`	MP4 download

Optional headers: `X-Api-Key` (if `Service:ApiKey` set), `X-ElevenLabs-Api-Key`, `X-OpenAI-Api-Key`, `X-Azure-OpenAI-Api-Key`.

7. Frontend / Internal UI Workings

There is no component hierarchy, router, form library, or rendering lifecycle in the SPA sense.

7.1 Console as UI

- **Banner** in ProductionApp: transcript, output, mode, `Harness : MAF+tools`, cleared file count, stub lines, TTS provider, **Memory path**.
- **Tool progress** in DirectExecutor: each tool return string WriteLine.
- **Agent stream** in WorkflowHost: executor id headers and incremental `update.Update.Text`.
- **Host logging** Microsoft.Extensions.Logging to console (default Host logging).

7.2 State management

UI is stateless. PipelineStore is the application state. Operators infer progress from logs and from files appearing under `output/`.

7.3 Event handling

Ctrl+C triggers Host `ApplicationStopping`. The harness passes that token into WorkflowHost and DirectExecutor; tools pass it into Ollama, ElevenLabs, and ffmpeg. Mutex is released when the process exits (using Dispose on Mutex).

7.4 Validation

CLI has no forms. Invalid transcript path → FileNotFoundException before work starts. n8n Form Trigger is the operator UI for the job API (§19); it is not hosted by this EXE.

8. Backend / Internal Service Workings

8.1 Business logic placement

Rule	Where enforced
Exactly N scripts (default 8)	TranscriptSegmenter + ProductionTools count check + SaveNarrationScripts + HarnessChecks
Max M slides (default 5)	SlidePlannerService clamp; RenderAllSlideImages Take(); HarnessChecks
Never segment raw transcript after rewrite exists	LoadNarrationSource / SegmentTranscriptIntoEightScripts

Slide on-screen text is one sentence	SlidePhraseRewriteService (Ollama or stub); PNG uses Phrase
Overwrite media on rerun; keep memory folders	PipelineStore.Initialize + tools always write
Single instance	named Mutex
Spoken talk length ~7 min	prompt only (Ollama); stub file is used as-is

8.2 Controllers / handlers

`ProductionApp.RunAsync` is the CLI entry. `JobEndpoints` is the HTTP entry. `ProductionHarness.RunAsync` is the production loop. Tool methods are the command handlers.

8.3 Middleware

Agent middleware: `ProductionGuardrails.RunAsync` (message filter) and `FunctionAsync` (tool log). Service mode also uses ASP.NET `ApiKeyMiddleware` (optional `X-Api-Key`). Host logging is standard.

8.4 Background jobs / queues / events

CLI: no queue. Service: `JobQueue` is a `Channel<string>`; `JobWorker` is a serial `BackgroundService` (one production at a time). ffmpeg is still synchronous from the job's perspective. MAF streaming is async enumeration on that job's scope.

8.5 External integrations

System	Used for	Failure mode
Ollama	Narration rewrite (if not stub); slide sentences (if not stub slides); default agent LLM	Timeout 30 min; slide rewrite is sequential per slide. Direct mode with both stubs on does not need Ollama.
OpenAI / Azure OpenAI	Optional agent LLM if keys/endpoint present	Takes priority over Ollama
ElevenLabs	TTS	Hard fail, no fallback
ffmpeg/ffprobe	MP3 encode, duration, slideshow, merge	Must be on PATH; 8 min timeout
Windows SAPI	TTS fallback if no ElevenLabs key	Falls back further to NAudio tone bed

9. Security

9.1 Authentication and authorization

No user login or roles. Anyone who can run the EXE as the current user can produce videos and read `output/`. The job API accepts unauthenticated requests when `Service:ApiKey` is empty (logs a warning). When set, clients send `X-Api-Key`. n8n Form Trigger is a public URL if the workflow is active — keep API keys on n8n credentials, not the form.

Outbound: ElevenLabs API key; optional OpenAI key; optional Azure key or DefaultAzureCredential (managed identity / VS / az cli — *inferred* Azure.Identity behavior).

9.2 Data protection

Transcripts and generated media sit unencrypted on disk under `output/` (gitignored). Notes in pipeline-state.json may contain agent excerpts. PII redaction in guardrails is regex-only on agent messages, not on transcript files or TTS payloads. Emails in the source talk are sent to ElevenLabs in full. Confirmed: redaction does not wrap ProductionTools in direct mode.

9.3 Input validation

Filesystem paths are combined from config and args without a sandbox. A transcript argument of an unexpected file will be read if it exists. ffmpeg concat lists include absolute paths. There is no command-injection via ArgumentList (good: `ProcessStartInfo.ArgumentList` is used, not a single shell string).

9.4 Session / tokens

No app sessions. Named mutex is a lock, not auth. HttpClients do not implement retry/auth refresh.

9.5 Attack vectors and mitigations

Vector	Situation	Mitigation / gap
Prompt injection via transcript	Agent modes send status + files to an LLM	Guardrail blocked terms are trivial; tools still execute if the model calls them. Direct mode ignores transcript as instructions.
Secret leakage	API key in git-tracked appsettings.json	**Gap.** User requested appsettings storage. Do not commit to a public remote. UserSecretsId exists but is unused for ElevenLabs.
SSRF	Model:Endpoint is operator-controlled	Acceptable for a local tool; do not bind endpoint to untrusted input.

Path traversal	OutputDirectory from config	No jail. Operator-trusted config.
Resource exhaustion	Huge transcript → huge TTS bills / long ffmpeg	No size cap besides Ollama/stub length check of 400 min, not max.
Overlapping ffmpeg locks	two runs	Mutex

9.6 Secrets and configuration management

Confirmed sources: JSON, environment (Host defaults + explicit ElevenLabs env fallback), user-secrets package referenced but not loaded with a dedicated call (Generic Host **does** add user secrets in Development *inferred* when DOTNET_ENVIRONMENT=Development). Current launch profile does not set environment. ElevenLabs key in Studio section is the live source.

This document does not reprint secret values. Rotate any key that was committed or pasted into chat.

10. Error Handling and Edge Cases

10.1 Propagation

Exceptions from services → tools → DirectExecutor or MAF tool invocation → ProductionHarness → ProductionApp catch logs Error and rethrows. After a "successful" tool loop, `HarnessChecks` can still throw. No global exception handler / EventId taxonomy beyond default logger categories.

10.2 Logging

ILogger on ProductionApp, ProductionHarness, DirectExecutor, AgentFactory, WorkflowHost, NarrationRewriteService, TtsService, ProductionGuardrails. Information for provider selection and ElevenLabs writes; Warning for stub rewrite, SAPI failure, agent create failure, incomplete agent merge, guardrail block; Error for pipeline failure and harness-check failure.

No structured file log, no Application Insights, no correlation ID beyond RunId in state/filenames.

10.3 Retries

Operation	Retry
Delete generated files	Up to 8 attempts, backoff 150ms × attempt
PNG write	6 attempts + sidecar copy
ffmpeg merge	One fallback re-encode, not a counted retry

ElevenLabs / Ollama HTTP	None

10.4 Failure scenarios

- **Second instance:** InvalidOperationException mutex.
- **Missing transcript / stub:** FileNotFoundException.
- **Stub too short:** InvalidOperationException.
- **Player has video_*.mp4 open:** Initialize throws IOException after retries.
- **ffmpeg missing:** Win32Exception / "failed to start ffmpeg".
- **ElevenLabs 401/quota:** InvalidOperationException with body; run aborts mid-audio (earlier MP3s already written this run; next run wipes them).
- **Ollama down in handoff:** TryCreate may still construct a client; failure appears when the first model call happens. Direct mode with both stubs on never needs Ollama.
- **Agent never merges:** failover DirectExecutor, then HarnessChecks.
- **Agent claims Done with missing files:** HarnessCheckException; process exits non-zero.

10.5 Validation failures

Wrong agent JSON for SaveNarrationScripts, empty transcript, missing audio before compose. All throw; no partial merge if fewer than SegmentCount videos.

11. Configuration and Environment

11.1 Files

File	Role
`src/NarrationStudio/appsettings.json`	Primary Service + Studio + Model settings; copied to output dir
`bin/Debug/net10.0-windows/appsettings.json`	What `dotnet run` actually binds when cwd is repo root
`appsettings.Development.json`	gitignored; optional if you add it
`Properties/launchSettings.json`	workingDirectory = repo root

11.2 Studio section (confirmed keys)

Key	Default in code	Meaning

SegmentCount	8	Scripts / MP3s / MP4s required
MaxSlidesPerSegment	5	Planner clamp and render trim
TargetTalkMinutes	7	Ollama word-budget; context instructions
UseStubNarration	true	Skip Ollama *narration* rewrite only. Independent of slide sentences.
UseStubSlidePhrases	false	If true, on-screen sentences are the planner excerpts (no Ollama).
StubNarrationPath	voice_transcript_ollama.txt	Relative stub file
TranscriptPath	empty	Optional explicit transcript
OutputDirectory	output	Relative to cwd unless rooted
OpenAIModel	gpt-4o-mini	If OPENAI_API_KEY set
AzureOpenAIEndpoint / Deployment	empty / gpt-4o-mini	Agent client if endpoint set
ElevenLabsApiKey	empty in class; set in JSON	TTS
ElevenLabsVoiceId	UPXQvvSDDubCtocmbltQ	TTS voice
ElevenLabsModelId	eleven_multilingual_v2	TTS model
VideoWidth / Height / FrameRate	1920 / 1080 / 30	PNG and ffmpeg vf

11.2b Service section

Key	Default	Meaning
ListenUrl	http://0.0.0.0:5088	Kestrel bind when not in CLI mode
ApiKey	empty	If set, required as `X-Api-Key` except `/health`. Mint a random string (no generate endpoint). Prefer `Service__ApiKey` env var so it is not committed. Same value goes in the n8n **Narration Studio API** credential. See §19 and `docs/n8n-service.pdf` §3.1.

11.3 Model section

Endpoint default http://localhost:11434; Model qwen3.8:latest; TimeoutMinutes 30. From n8n these are host URLs (Windows), not addresses inside the n8n container.

11.4 Environment variables

Variable	Effect
ELEVENLABS_API_KEY	Used only if JSON key is empty
OPENAI_API_KEY / OPENAI_MODEL	Agent chat client
AZURE_OPENAI_ENDPOINT / API_KEY / DEPLOYMENT_NAME / DEPLOYMENT	Azure agent client; endpoint wins over OpenAI and Ollama
DOTNET_ENVIRONMENT	Standard Host; not set by launch profile

11.5 Feature flags

JSON flags: `UseStubNarration` and `UseStubSlidePhrases` (independent). Orchestration mode is a CLI flag or a job-request field (n8n jobs default to `direct`). No staging/production splits exist in-repo.

12. Deployment and Infrastructure

Confirmed: Studio itself is a local Windows executable. n8n that calls it is an existing Docker stack on the same PC, not a container of this repo.

- **Build:** `dotnet build` / Visual Studio 2026 on `NarrationStudio.slnx`. SDK-style csproj, TargetFramework net10.0-windows.
- **CLI:** `dotnet run --project src\NarrationStudio -- --orchestration direct` from repo root.
- **Job API:** `dotnet run --project src\NarrationStudio` (empty args) listens on `0.0.0.0:5088`. Only one process may lock the exe.
- **Hosting:** operator's Windows 10/11 machine. Requires ffmpeg/ffprobe on PATH, optional Ollama, optional network to ElevenLabs.
- **n8n:** Docker compose project `n8n-api_icountera`. This repo mounts `n8n/n8n-nodes-narration-studio` read-only. Exact YAML and commands: §19 and `docs/n8n-service.pdf`.
- **Health:** `GET /health` when the job API is running. No GitHub Actions, no k8s, no Dockerfile for Studio.
- **Publish:** `dotnet publish` would produce a framework-dependent or self-contained exe; not scripted here. System.Speech and Windows TFM mean Linux containers are not a supported Studio target without replacing SAPI.

Observability is console logs plus per-job `events.jsonl`. Operators watch `output/` or `output/jobs/{id}/`.

13. Performance and Scalability

Stage	Cost	Notes

Stub narration rewrite	Negligible	File copy + strip
Ollama narration rewrite	Minutes; up to 30 min timeout	Only if UseStubNarration is false; logs elapsed seconds
Ollama slide sentences	**Usually the longest stage**	One sequential HTTP call per slide (up to 40). Observed tens to 160+ seconds each on local qwen3.8. Logs per-slide and total elapsed time.
Segment / regex plan	Milliseconds	Before Ollama sentences
PNG render	Seconds for ~40 images	Draws the Ollama sentence
ElevenLabs TTS	High when enabled	8 sequential HTTP calls, 3 min timeout each worst case
ffmpeg compose	High CPU	8 sequential encodes 1920×1080
ffmpeg merge copy	Fast	Re-encode fallback is slow

Concurrency. No parallel TTS or ffmpeg. Mutex forbids two pipelines. The job API queues HTTP jobs one at a time through `JobWorker`.

Scalability. Designed for one talk, one machine, one run. Not multi-tenant. n8n can submit many form posts; they wait in the channel. Current bottleneck with narration stubbing on and slide stubbing off is **per-slide Ollama**; with ElevenLabs enabled, TTS is the other long tail. Both stubs on, TTS is the long tail.

Caching. Intentionally disabled for media so reruns match current narration.

14. Testing

Confirmed: `tests/NarrationStudio.Tests` is an xUnit project. Current coverage:

- `HarnessChecksTests` — pass when artifacts exist; fail when merged video is missing.
- `TranscriptSegmenterTests` — configured segment count.
- `SlidePlannerTests` — never exceeds max slides.
- `StudioNamingTests` — audio/video ranges follow SegmentCount.
- `JobOptionsMergerTests` / `JobPathsTests` / `JobRequestJsonTests` — overlay, isolated job folders, string bools/ints from n8n JSON.

There is no end-to-end test that calls Ollama, ElevenLabs, or ffmpeg. Outer agent harness for this repo: `AGENTS.md` and `.cursor/rules/studio-harness.mdc`.

Manual matrix a developer should still run:

- direct + stub narration + Ollama up → slide PNGs show one-sentence rewrites; memory folder has run.txt / pipeline-status.txt / notes.txt / events.jsonl

- direct + both stubs → no Ollama required; HarnessChecks pass if media exists
- direct + stub + ElevenLabs → N MP3s + merged mp4
- direct + stub + no API key → SAPI path
- UseStubNarration false with Ollama up → narration-rewritten.txt from model
- handoff: MAF FileMemoryProvider constructed; snapshots still written; missing merge fails over then still runs checks
- rerun keeps old production-notes-* folders and deletes previous video_*.mp4
- second instance mutex
- locked output file → clear failure message

Run tests: `dotnet test tests\NarrationStudio.Tests`.

15. Developer Guide

15.1 How to understand the codebase

Read in this order: `Program.cs` → `ProductionApp.cs` → `Harness/ProductionHarness.cs` → `DirectExecutor.cs` → `ProductionTools.cs` → the service for the stage you care about → only then `AgentFactory` / `WorkflowHost`. The harness owns the loop; agents are a control plane over tools; tools are the product.

15.2 Where functionality lives

If you want to change…	Start here
CLI vs Kestrel branch	Program.cs
CLI, mutex, banner, mode default	ProductionApp
HTTP jobs, queue, options overlay	Service/JobEndpoints, JobWorker, JobOptionsMerger
n8n node / form	n8n/n8n-nodes-narration-studio, n8n/workflows/narration-studio-form.json
Stub copy, MAF vs direct, failover, checks	ProductionHarness
events.jsonl	HarnessEvents
Process-level Done gate	HarnessChecks
Timestamped memory folder	PipelineStore.AgentMemoryDirectory
Stub vs Ollama rewrite	NarrationRewriteService, StudioOptions

How text is chopped into N	TranscriptSegmenter
On-screen slide sentences	SlidePhraseRewriteService (Ollama or stub); planner only splits
Slide look	SlideImageRenderer
Voice / TTS vendor	TtsService, appsettings
Video timing / codecs	VideoComposerService
Agent personalities / tool grants	AgentFactory
Handoff graph	WorkflowHost
Blocked words	ProductionGuardrails

15.3 Extending

- **Ninth segment:** raise SegmentCount; prompts, tool descriptions, and `StudioNaming` interpolate the count. Re-run tests.
- **New specialist:** add tool method, wrap in AgentFactory, add handoff edges, update sequential list and coordinator text.
- **Cloud LLM rewrite:** swap OllamaChatClientFactory usage in NarrationRewriteService; keep stub flag.
- **Stable output name:** today merged name includes RunId; consumers must read the Done line or pipeline-state.json.

15.4 Common workflows

```
cd E:\exercises\maf
dotnet build
dotnet run --project src\NarrationStudio -- --orchestration direct
dotnet run --project src\NarrationStudio
# second command is the n8n job API; do not run both at once
```

Edit transcripts at repo root. Inspect `output/scripts/narration-rewritten.txt` (CLI) or `output/jobs/{id}/` (n8n). Inspect `output/agent-file-memory/production-notes-*` for `run.txt`, status, and `events.jsonl`. Play the newest `video_*.mp4`.

15.5 Pitfalls and technical debt

- appsettings loaded from bin copy; editing source JSON requires rebuild/copy.
- Secrets in tracked JSON.
- Guardrails do not apply to direct mode.
- SummarizeForAgent next-step order prefers audio before remaining slide plans if scripts exist — odd if someone called TTS without planning (direct mode does not do that).

- No end-to-end tests against Ollama/ElevenLabs/ffmpeg.
- Tone-bed fallback can produce a video that “works” with a beep instead of speech if SAPI fails and no ElevenLabs key.
- FileMemoryProvider types come from MAF; upgrading Microsoft.Agents.AI can break AgentFactory.

16. Complete Dependency Map

16.1 Internal

```
Program
  └─ ProductionApp
        ├─ StudioOptions, TtsService (banner)
        ├─ PipelineStore
        └─ ProductionHarness
              ├─ ProductionTools ──┬─ NarrationRewriteService — OllamaChatClientFactory
              │                    ├─ TranscriptSegmenter
              │                    ├─ SlidePlannerService
              │                    ├─ SlidePhraseRewriteService — OllamaChatClientFactory
              │                    ├─ SlideImageRenderer
              │                    ├─ TtsService — FfmpegProcess, NAudio, System.Speech, HttpClient
              │                    ├─ VideoComposerService — FfmpegProcess
              │                    └─ HarnessEvents (events.jsonl)
              ├─ AgentFactory ──────── ProductionTools, PipelineStore, ProductionContextProvider,
              │                        OllamaChatClientFactory, ProductionGuardrails (new per TryCreate),
              │                        FileMemoryProvider (package)
              ├─ WorkflowHost ──────── PipelineStore, ProductionCrew
              ├─ DirectExecutor ─── ProductionTools, PipelineStore
              └─ HarnessChecks
```

16.2 NuGet packages (csproj) and why

Package	Why	Where
Microsoft.Extensions.Hosting 10.0.0	Generic host, DI, logging, config	Program.cs
Microsoft.Extensions.Configuration.*	JSON, env, user secrets	Program.cs, csproj
Microsoft.Agents.AI 1.17.0	AIAgent, AsAIAgent, context, file memory types	AgentFactory, Guardrails, Memory
Microsoft.Agents.AI.Workflows 1.17.0	Handoff/sequential workflows, InProcessExecution	WorkflowHost
Microsoft.Agents.AI.OpenAI 1.17.0	OpenAI/Azure IChatClient adapters	AgentFactory

Microsoft.Extensions.AI 10.7.0	IChatClient, ChatMessage, AIFunctionFactory	Agents, rewrite, tools wrap
Microsoft.Extensions.AI.OpenAI 10.6.0	AsIChatClient extensions	AgentFactory
Azure.AI.OpenAI 2.1.0 / Azure.Identity 1.14.2	Azure client + DefaultAzureCredential	AgentFactory
OpenAI (transitive via packages)	OpenAIClient	AgentFactory
OllamaSharp 5.4.25	OllamaApiClient	OllamaChatClientFactory, used by narration rewrite and slide sentences
SixLabors.ImageSharp 3.1.11 + Drawing + Fonts	PNG slides	SlideImageRenderer
NAudio 2.2.1	Tone-bed fallback WAV	TtsService.GenerateToneBed
System.Speech 9.0.8	Windows TTS	TtsService

NoWarn includes MAAI001, OPENAI001, MEAI001 — experimental / preview analyzer IDs for MAF/OpenAI/MEAI.

16.3 External runtime dependencies

ffmpeg, ffprobe, Windows, optional Ollama daemon, optional ElevenLabs network, optional Azure/OpenAI network.

17. Technical Diagrams

17.1 Overall architecture

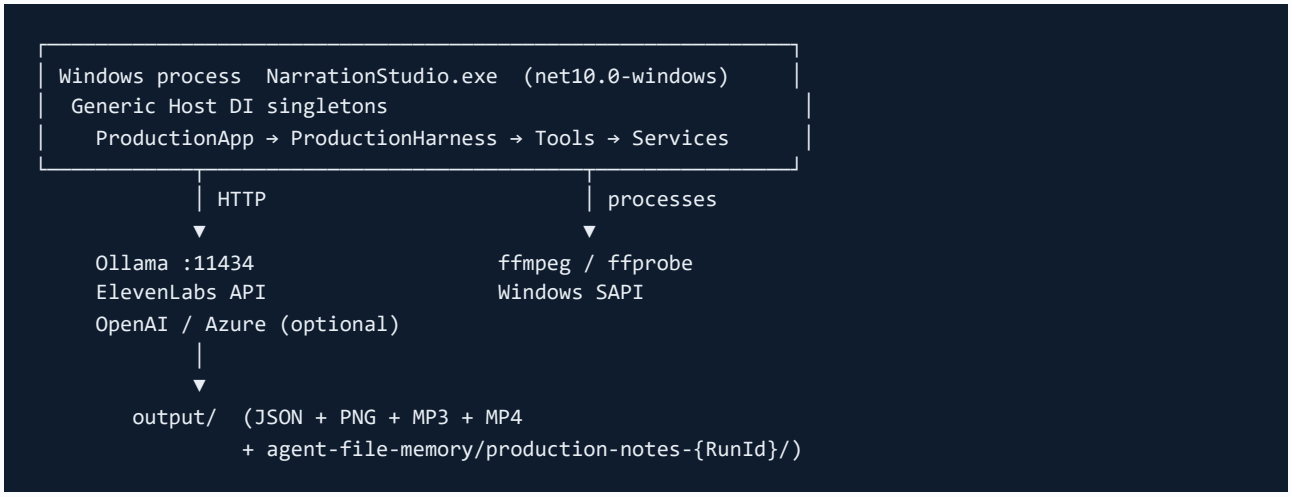


Figure 2. Process and externals. Confirmed.

17.2 Component relationships

See §16.1. All arrows are constructor injection except ffmpeg (Process.Start) and HTTP clients created inside services.

17.3 Data flow



Figure 3. Artifact lineage. Later stages must not read the raw transcript once narration-rewritten.txt exists.

17.4 Direct-mode lifecycle

See §4.1. Sequence is strictly linear. Plan and render are async (Ollama). Each tool Mutate persists JSON and refreshes the timestamped memory snapshots. Render skips Ollama when `PhraseRewritten` is already true.

17.5 Handoff lifecycle

```
User prompt
   → coordinator (GetPipelineStatus)
        ├─[stub off]→ narration_rewriter → Rephrase… → script_segmenter
        └─[stub on]→ script_segmenter → LoadNarrationSource + Segment…
             → coordinator
             → slide_planner → PlanSlides… (Ollama or stub sentence per slide) → image_renderer
             → coordinator
             → tts_producer → SynthesizeAllAudioAsync
             → coordinator
             → video_composer → Compose… + Merge…
             → coordinator stop when merged exists
   → if no merged file: DirectExecutor (full remaining tools)
   → HarnessChecks.Verify
```

Figure 4. Handoff control. Routing is model-directed; tools perform I/O.

17.6 Data “ER” diagram

```
PipelineState
   runId PK (string, time-based)
   1 ──< NarrationSegment.number  (1..SegmentCount)
              1 ──< SlideSpec.number (1..≤5)
                    sourcePhrase, phrase, phraseRewritten
   notes[]  (capped 24)
```

Figure 5. In-memory / JSON relationship. No RDBMS.

17.7 Authentication flow

```
Operator runs EXE as Windows user
   → no login
   → TtsService.ResolveApiKey: JSON then env
        → if set: xi-api-key on ElevenLabs POST
   → AgentFactory.CreateChatClient:
        Azure endpoint? → ApiKeyCredential or DefaultAzureCredential
        else OPENAI_API_KEY? → OpenAIClient
        else Ollama (no auth in this repo)
```

Figure 6. Outbound credential flow only.

17.8 Deployment architecture

Single box: developer workstation. No load balancer, no container, no CDN. Distribution, if any, is copying the published folder plus ensuring ffmpeg and appsettings are present.

18. Appendix

18.1 Glossary

Term	Meaning in this codebase
MAF	Microsoft Agent Framework (NuGet Microsoft.Agents.AI*)
Direct mode	No agents; DirectExecutor calls tools in order (also the failover executor)
Harness	ProductionHarness + checks + events; owns the run loop this app implements
Handoff	Coordinator routes to specialist agents (MAF)
Stub narration	Pre-written talk in voice_transcript_ollama.txt used instead of Ollama *narration* rewrite. Independent of slide stubbing.
Stub slide phrases	UseStubSlidePhrases copies SourcePhrase to Phrase without Ollama
SourcePhrase / Phrase	Planner excerpt vs on-screen sentence drawn on the PNG
RunId	Local timestamp used in merged filename
ProductionTools	The in-process API agents and direct mode share
Guardrails	Agent middleware for blocked terms and PII regex; not a full policy engine
SAPI	Windows Speech API used when no ElevenLabs key

18.2 Important files and directories

```
E:\exercises\maf\
  NarrationStudio.slnx
  README.md
  voice_transcript.txt              raw source notes
  voice_transcript_ollama.txt       stub 7-minute talk
  output\                           gitignored generated tree
  src\NarrationStudio\
    Program.cs
    ProductionApp.cs
    appsettings.json
    NarrationStudio.csproj
    Harness\ProductionHarness.cs
    Harness\HarnessChecks.cs
    Harness\HarnessEvents.cs
    Agents\AgentFactory.cs
    Agents\WorkflowHost.cs
    Orchestration\DirectExecutor.cs
    Tools\ProductionTools.cs
    Services\SlidePhraseRewriteService.cs
    Models\PipelineModels.cs
    Configuration\*.cs
    Guardrails\ProductionGuardrails.cs
    Memory\ProductionContextProvider.cs
    Properties\launchSettings.json
    Service\JobEndpoints.cs
    Service\JobWorker.cs
    Hosting\StudioServiceCollection.cs
  n8n\
    n8n-nodes-narration-studio\
    workflows\narration-studio-form.json
    docker-compose.n8n-excerpt.yml
  tests\NarrationStudio.Tests\
  AGENTS.md
  .cursor\rules\studio-harness.mdc
  docs\
    NarrationStudio-Internal-Workings.html / .pdf   this document
    n8n-service.md / .html / .pdf
    memory-vs-mem0.md / .pdf
    guardrails.md / .pdf
    harness-engineering.md / .pdf
    agentic-system.md / .pdf
    orchestration-modes.md / .pdf
```

18.3 Configuration reference

See §11. Do not copy secrets into documentation, tickets, or screenshots.

18.4 Key assumptions

- Operator is on Windows with .NET 10 SDK and ffmpeg on PATH.
- One talk at a time; output directory is expendable.
- Transcript language works with the chosen TTS model/voice.

- Default run uses stub narration, live Ollama for slide sentences unless `UseStubSlidePhrases`, and ElevenLabs if `Studio:ElevenLabsApiKey` (or env) is set.
- Repo root is the working directory so relative output/ and transcripts resolve.

18.5 Known limitations

- No SPA, SQL, CI, or multi-run resume. Job API + n8n form are the HTTP surface. Unit tests cover checks/segmenter/planner/naming/job merge, not live media APIs.
- Agent quality depends on the LLM; direct mode is the reliable factory; HarnessChecks is the Done gate.
- Secrets currently live in appsettings.json.
- No max transcript size; ElevenLabs character limits can fail a segment.
- Slide on-screen text depends on Ollama unless `UseStubSlidePhrases` is true.
- Slide timing is even split of audio duration, not word-level alignment.
- Guardrails do not apply to direct mode. Caps are enforced in tools and HarnessChecks.

18.6 Areas requiring further investigation

- Exact MAF handoff / autonomous turn semantics on package 1.17.0 (only usage sites are in WorkflowHost).
- FileMemoryProvider on-disk layout during handoff (direct mode only writes Narration Studio snapshots).
- Whether Host user-secrets load without Development environment.
- ElevenLabs voice settings (stability/similarity) — currently only text + model_id.
- ffprobe duration locale parsing (uses current-culture TryParse, not InvariantCulture) — possible issue on some locales.

19. n8n job service

This section is how n8n is used with Narration Studio. Operator YAML and copy-paste compose commands live in `docs/n8n-service.pdf` (source `docs/n8n-service.html` / `docs/n8n-service.md`).

19.1 Why n8n is outside the factory

n8n already runs in Docker on this PC (stack `n8n-api_icountera`, UI `http://localhost:5678`). The container cannot browse `E:\…`. Studio stays a Windows process so ffmpeg, SAPI, and Ollama on `localhost:11434` keep working. n8n only starts a job, waits, and downloads the MP4. It does not write slides or audio.

A full run exceeds n8n's default HTTP timeout, so `POST /v1/jobs` returns **202** immediately. `JobWorker` runs productions one at a time under `output/jobs/{jobId}/`. Form values overlay `appsettings.json` defaults via `JobOptionsMerger`. API keys stay on n8n credentials, not the form.

19.2 How a form submission runs

1. Operator opens the Form Trigger URL and uploads transcript + stub narration (browse). Other fields are pre-filled from `appsettings.json`.
2. The custom node (`n8n-nodes-narration-studio`) POSTs JSON to `http://host.docker.internal:5088/v1/jobs` with real booleans/numbers and file text as `transcriptText` / `stubNarrationText`. Form binaries are often named `Transcript_file` / `Stub_narration_file`.
3. Studio writes inputs, enqueues the job, returns `jobId`.
4. The node polls `GET /v1/jobs/{id}` until `complete` or `failed`, then downloads `/v1/jobs/{id}/artifacts/merged`.
5. Inside the job, `ProductionHarness` + `ProductionTools` run exactly as in the CLI. n8n jobs default orchestration to `direct`.

19.3 Docker Compose file (this PC)

Live: `E:\icountera\n8n-api_icountera\n8n\docker-compose.yml`. Sanitized copy with ADDED comments: `n8n/docker-compose.n8n-excerpt.yml`. Always `-p n8n-api_icountera`.

Already there: postgres:16, n8n:latest on 5678, `.env` secrets, `n8n_data`. **Added:** `N8N_CUSTOM_EXTENSIONS`, `N8N_UNVERIFIED_PACKAGES_ENABLED`, `extra_hosts: host.docker.internal:host-gateway`, volume mount of `n8n-nodes-narration-studio`, then `npm install` in `/home/node/.n8n/nodes`.

```
      N8N_CUSTOM_EXTENSIONS: /custom-nodes/n8n-nodes-narration-studio   # ADDED
      N8N_UNVERIFIED_PACKAGES_ENABLED: "true"                          # ADDED
    extra_hosts:
      - "host.docker.internal:host-gateway"                            # ADDED
    volumes:
      - n8n_data:/home/node/.n8n
      - E:/exercises/maf/n8n/n8n-nodes-narration-studio:/custom-nodes/n8n-nodes-narration-studio:ro  #
ADDED
```

```
cd E:\icountera\n8n-api_icountera\n8n
docker compose -p n8n-api_icountera up -d
docker compose -p n8n-api_icountera exec n8n sh -c "mkdir -p /home/node/.n8n/nodes && cd
/home/node/.n8n/nodes && npm install /custom-nodes/n8n-nodes-narration-studio"
docker compose -p n8n-api_icountera restart n8n
```

Update this node: edit `n8n/n8n-nodes-narration-studio/dist/`, then `restart n8n` + Ctrl+Shift+R. Re-`npm install` only if `package.json` changed.

Add another n8n node (not a MAF agent): new folder `n8n/n8n-nodes-<name>`, extra volume, colon-separate `N8N_CUSTOM_EXTENSIONS`, `up -d`, `npm install` that path, restart. Do not mount the whole `n8n/` directory. MAF specialists still go in `src/NarrationStudio/Agents/`.

Do **not** use n8n's "Install community nodes" dialog for this package name — that hits the public npm registry.

19.3b Job API key

There is no generate-key endpoint. Mint a secret in PowerShell, e.g. `[Convert]::ToBase64String((1..32 | ForEach-Object { Get-Random -Maximum 256 }) -as [byte[]])`, or a GUID. Put it in the **bin** `appsettings.json` `Service:ApiKey`, or set `$env:Service__ApiKey` and restart Studio. Until a key is set, the node may run with no credential. Full click-path: `docs/n8n-service.pdf` §3.1.

A **Narration Studio API** credential is one n8n vault record: **Base URL** `http://host.docker.internal:5088` (not `localhost` inside Docker); **API Key** = the same string as `Service:ApiKey`, sent as `X-Api-Key`; optional ElevenLabs / OpenAI / Azure fields are *other* headers, not this key. Create it under Credentials → Add credential → search Narration Studio API (mounted node, not public npm). **Test** calls `/health` and only proves the URL is up.

Attach that credential on **Start job**, **Wait until complete**, and **Download merged video**. If only Start job has it, the later nodes get 401. Do not attach it to **On form submission**. The form is shareable; the job key is server-to-server (n8n encrypts the vault with `N8N_ENCRYPTION_KEY`). An ElevenLabs key in the **API Key** field will not unlock TTS and will 401 if it does not match `Service:ApiKey`. After rotating the Studio key, restart `dotnet run` and edit the credential.

19.4 What broke on first use

- Missing credentials → credentials made optional; default base URL `host.docker.internal:5088`.
- Start job 500 → n8n sent string `"true"` for booleans. Studio now accepts string bools/ints; parse errors are 400.
- MSB3027 → two `dotnet run` instances locking the exe.

End of document. Revision 4, generated from the Narration Studio source tree as of 22 August 2026. If the code and this PDF diverge, the code wins.